

Dockerfile Avanzado

Arturo Silvelo

Try New Roads

Networking

El subsistema de red de Docker es **modular**, utilizando drivers. Existen varios drivers por defecto que proporcionan funcionalidad de red principal:

- **Bridge Network:** La red por defecto.
- **Host Network:** Elimina el aislamiento.
- **None Network:** Aislamiento total.
- **Overlay Network:** Conectar multiple Docker daemons.

Bridge network

Bridge network

Permite que los contenedores conectados a la misma red se comuniquen, mientras proporciona aislamiento de contenedores que no están conectados a esa red.

Default vs user-defined

- Las `user-defined` proporcionan resolución automática de DNS entre contenedores:

Los contenedores en la red bridge por defecto solo pueden acceder entre sí por direcciones IP, a menos que uses la opción `--link`, que se considera obsoleta.

```
docker compose -f 04-network/ejemplos/1.isolated/compose.yml up -d
```

```
$docker exec app-base-1 ping app-base-2  
ping: bad address 'app-base-2'
```

- Las `user-defined` proporcionan un mejor aislamiento

```
docker compose -f 04-network/ejemplos/1.isolated/compose.yml up -d
docker compose -f 04-network/ejemplos/1.isolated/compose-p2.yml up -d
```

```
$docker inspect bridge --format='{{range .Containers}}{{.Name}} - {{.IPv4Address}}{{"\n"}}{{end}}'
```

app-base-3	-	172.17.0.4/16
app-base-1	-	172.17.0.3/16
app-base-2	-	172.17.0.2/16
app-base-4	-	172.17.0.5/16

- Las `user-defined` permiten una configuración específica.

Opciones de configuración Bridge

Opción	Por defecto	Descripción
<code>com.docker.network.bridge.enable_icc</code>	<code>true</code>	Habilitar/deshabilitar conectividad inter-contenedor
<code>com.docker.network.bridge.enable_ip_masquerade</code>	<code>true</code>	Habilitar acceso a internet (NAT)
<code>com.docker.network.driver.mtu</code>	<code>0</code> (sin límite)	MTU máximo de la red
<code>com.docker.network.bridge.host_binding_ipv4</code>	todas las IPs	IP del host por defecto al publicar puertos
<code>com.docker.network.bridge.name</code>	-	Nombre personalizado del bridge

- Red con comunicación inter-contenedor deshabilitada:

```
docker network create \  
  --opt com.docker.network.bridge.enable_icc=false \  
  --opt com.docker.network.bridge.name=red-aislada \  
  red-segura
```

- Red sin acceso a internet

```
docker network create \  
  --opt com.docker.network.bridge.enable_ip_masquerade=false \  
  red-local
```

- Verificar acceso a internet pero no entre contenedores

```
docker run --network red-segura --name test1 -d alpine sleep 300  
docker run --network red-segura --name test2 -d alpine sleep 300  
docker exec test1 ping test2  
docker exec test1 ping google.com
```

- Cambiamos las redes

```
docker network disconnect red-segura test1  
docker network disconnect red-segura test2  
docker network connect red-local test1  
docker network connect red-local test2
```

- Verificar acceso entre contenedores pero no a internet

```
docker exec test1 ping test2  
docker exec test1 ping google.com
```

IPAM (IP Address Management)

IPAM controla la asignación de direcciones IP en las redes Docker.

Opción	Descripción
<code>--subnet</code>	Rango de IPs de la red (CIDR)
<code>--gateway</code>	IP del gateway de la red
<code>--ip-range</code>	Subconjunto de IPs asignables
<code>--aux-address</code>	IPs reservadas (no asignables)

- Creación red

```
docker network create \
  --opt com.docker.network.bridge.name=red-ipam \
  --subnet=192.168.100.0/24 \
  --ip-range=192.168.100.128/25 \
  --gateway=192.168.100.1 \
  --aux-address="proxy=192.168.100.20" \
  --aux-address="dns=192.168.100.10" \
  custom-ipam
```

- Verificación

```
docker run --network custom-ipam alpine ip addr
```

- Verificación (Compose)

```
docker exec app-base-1 ip addr
docker exec app-base-2 ip addr
```

- compose.yaml

```
x-app-base: &app-base
build:
  context: ../../../../app
  dockerfile: ../04-volumes/ejemplos/Dockerfile
ports:
  - "3000:3000"
image: app-base
container_name: app-base
environment:
  NODE_ENV: development
  PORT: 3000
services:
  app-base-1:
    <<: *app-base
    container_name: app-base-1
    hostname: app-base-1

  app-base-2:
    <<: *app-base
    container_name: app-base-2
    hostname: app-base-2
    ports:
      - "3001:3000"

networks:
  default:
    name: compose-ipam
    ipam:
      driver: default
      config:
        - subnet: 192.168.120.0/24
          ip_range: 192.168.120.128/25
          gateway: 192.168.120.1
          aux_addresses:
            proxy: 192.168.120.20
            dns: 192.168.120.10
    driver_opts:
      com.docker.network.bridge.name: compose-ipam
      com.docker.network.bridge.enable_icc: "true"
```

Host network

Host network

La red de ese contenedor **no está aislada del host Docker** (el contenedor comparte el namespace de red del host), y el contenedor no obtiene su propia dirección IP asignada.

Ventajas

- Optimizar el rendimiento
- Situaciones donde un contenedor necesita manejar un gran rango de puertos

```
$docker run --rm --net=host app-base  
$curl -X POST http://localhost:3000/upload -F "file=@/home/silvelo/Downloads/lenna.png" -H "Content-Type: multipart/form-data"
```

None network

None network

Si quieres **aislar completamente** un contenedor, puedes usar la red `--network none` al iniciar el contenedor. Dentro del contenedor, **solo se crea el dispositivo loopback.**

```
docker run --rm --network none alpine:latest ip link show
```

Overlay network

Overlay network

Una red distribuida entre **múltiples hosts Docker daemon**. Esta red se sitúa encima de (overlays) las redes específicas del host, permitiendo que los contenedores conectados a ella se comuniquen de forma segura cuando el cifrado está habilitado.

Docker **maneja transparentemente el enrutamiento** de cada paquete hacia y desde el host Docker daemon correcto y el contenedor de destino correcto.

```
docker network create -d overlay --attachable my-attachable-overlay
```